

Software Requirements Specification

University Management System

Microservices Architecture

Version 1.0 • 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the University Management System (UMS). It serves as a reference for the development team and any stakeholders reviewing the system.

1.2 Scope

The UMS is a microservices-based platform designed to manage the academic lifecycle of a university. It covers user identity and access management, course and enrollment management, announcements, feedback, departmental structure, audit logging, real-time notifications, course chat, and a BFF (Backend-for-Frontend) dashboard layer.

1.3 Team

Name	Role
Hanya Wael	Team Member
Mariam Ashraf	Team Member
Jana Khaled	Team Member
Mahmoud Alaa	Team Leader
Ahmed Hany	Team Member
Yousef Ashraf	Team Member

2. System Overview

The University Management System is composed of four backend microservices and a React-based frontend:

- IAM Service (port 8081) — Handles registration, login, JWT issuance, and all user/student/teacher profile management.
- Academic Core Service (port 8082) — Manages courses, enrollments, announcements, feedback, departments, and audit logs.
- Communication Service — Manages notifications (REST + WebSocket push) and course chat messages (REST + STOMP WebSocket).
- API Gateway — Routes all client requests, enforces CORS, and hosts the BFF Dashboard aggregation layer.
- Eureka Server — Service registry enabling all microservices to discover each other.

Services communicate synchronously via REST (through the API Gateway or direct Feign/WebClient calls) and asynchronously via Apache Kafka for event-driven flows such as notification triggers.

3. Functional Requirements

ID	Use Case	Actor	Service
UC-01	Register	Public	IAM
UC-02	Login	Public	IAM
UC-03	View own profile	Authenticated	IAM
UC-04	View user by ID	Authenticated	IAM
UC-05	View all users	Authenticated	IAM
UC-06	Filter users by role	Admin	IAM
UC-07	Update user profile	Authenticated	IAM
UC-08	Activate user account	Admin	IAM
UC-09	Deactivate user account	Admin	IAM
UC-10	Delete user	Admin	IAM
UC-11	View all students	Authenticated	IAM
UC-12	View student profile details	Authenticated	IAM
UC-13	View student basic info	Authenticated	IAM
UC-14	View all teachers	Authenticated	IAM
UC-15	View teacher profile details	Authenticated	IAM
UC-16	View teacher basic info	Authenticated	IAM
UC-17	Create course	Teacher	Academic Core
UC-18	Update course	Teacher	Academic Core
UC-19	Delete course	Teacher	Academic Core
UC-20	View course details	Authenticated	Academic Core
UC-21	View all courses	Authenticated	Academic Core
UC-22	View popular courses	Authenticated	Academic Core
UC-23	Browse courses by department	Authenticated	Academic Core
UC-24	Search courses by teacher name	Authenticated	Academic Core
UC-25	View own courses (by teacher)	Teacher	Academic Core
UC-26	Enroll in course	Student	Academic Core
UC-27	Drop course	Student	Academic Core
UC-28	Remove enrollment by ID	Authenticated	Academic Core
UC-29	View own enrollments	Student	Academic Core

ID	Use Case	Actor	Service
UC-30	View students enrolled in course	Teacher	Academic Core
UC-31	Check specific enrollment	Authenticated	Academic Core
UC-32	Create course announcement	Teacher	Academic Core
UC-33	View announcements for a course	Authenticated	Academic Core
UC-34	View my announcements (student)	Student	Academic Core
UC-35	View my announcements (teacher)	Teacher	Academic Core
UC-36	Submit feedback on a course	Student	Academic Core
UC-37	View all feedback	Admin	Academic Core
UC-38	View recent feedback	Admin	Academic Core
UC-39	View feedback for a course	Authenticated	Academic Core
UC-40	View own feedback history	Authenticated	Academic Core
UC-41	Create department	Admin	Academic Core
UC-42	View all departments	Authenticated	Academic Core
UC-43	View all audit logs	Admin	Academic Core
UC-44	Filter audit logs by action type	Admin	Academic Core
UC-45	View last-week logs by role	Admin	Academic Core
UC-46	View notification details	Authenticated	Communication
UC-47	View all notifications (inbox)	Authenticated	Communication
UC-48	View unread notifications	Authenticated	Communication
UC-49	Count unread notifications	Authenticated	Communication
UC-50	Filter notifications by type	Authenticated	Communication
UC-51	Mark notification as read	Authenticated	Communication
UC-52	Mark all notifications as read	Authenticated	Communication
UC-53	Delete a notification	Authenticated	Communication
UC-54	Delete all my notifications	Authenticated	Communication
UC-55	Send notification to a user	Teacher	Communication
UC-56	Send notification to course	Teacher	Communication
UC-57	Send message in course chat (REST)	Authenticated	Communication
UC-58	Send message via WebSocket	Authenticated	Communication
UC-59	View course chat history	Authenticated	Communication
UC-60	View messages sent by a user	Authenticated	Communication

ID	Use Case	Actor	Service
UC-61	Count messages in a course	Authenticated	Communication
UC-62	Delete a message	Authenticated	Communication
UC-63	View student dashboard	Student	API Gateway
UC-64	View teacher dashboard	Teacher	API Gateway
UC-65	View current user dashboard	Authenticated	API Gateway

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API responses must return within 2 seconds under normal load.
NFR-02	Scalability	Each microservice must scale independently via Docker/container orchestration.
NFR-03	Security	All endpoints (except /auth/**) require a valid JWT. Passwords hashed with BCrypt.
NFR-04	Availability	System targets 99.9% uptime; Eureka provides service discovery and failover.
NFR-05	Maintainability	Codebase follows SOLID principles and Hexagonal Architecture per service.
NFR-06	Usability	Frontend built with React (Vite); dashboard loads in under 3 seconds.
NFR-07	Auditability	All critical operations are logged via the Audit Log service.

5. Design Constraints

ID	Area	Constraint
CON-01	Language	Java 17 (Spring Boot 3) for all backend microservices.
CON-02	Frontend	React 18 with Vite as the build tool.
CON-03	Database	Relational DB (MySQL/PostgreSQL) initialized via init.sql.
CON-04	Auth	JWT-based stateless authentication; no session cookies.
CON-05	Messaging	Apache Kafka for asynchronous event-driven communication between services.
CON-06	Real-time	STOMP over WebSocket (Spring WebSocket) for live course chat.
CON-07	Deployment	Docker Compose orchestrates all services; Eureka for service registry.

6. Actors

6.1 Public (unauthenticated)

Any user who has not yet logged in. May only access `/api/auth/register` and `/api/auth/login`.

6.2 Student

A registered user with `ROLE_STUDENT`. Can enroll in and drop courses, view their dashboard, read announcements addressed to them, submit feedback, view and manage their notifications, and participate in course chat.

6.3 Teacher

A registered user with `ROLE_TEACHER`. Can create, update, and delete courses they own, post announcements, view enrolled students, send notifications to users or entire courses, and manage course chat.

6.4 Admin

A privileged user with `ROLE_ADMIN`. Has full control over user accounts (activate, deactivate, delete), can view all feedback and audit logs, and can create departments.

6.5 Authenticated

Any logged-in user regardless of role. Has access to general read operations such as viewing courses, departments, notifications, and enrollment status.

7. API Gateway & Service Discovery (Eureka)

7.1 Eureka Service Registry

The system uses Netflix Eureka for service discovery. The Eureka Server runs on port 8761 and is the first service that must start. Every other microservice registers itself with Eureka on startup using its `spring.application.name` as the service identifier. The API Gateway resolves upstream service addresses using the `lb://` (load-balanced) URI scheme, so no hardcoded IP addresses are needed in production.

- Eureka Server port: 8761
- IAM Service registers as: IAM-SERVICE
- Academic Core registers as: ACADEMIC-CORE
- Communication Service registers as: COMMUNICATION-SERVICE
- API Gateway registers as: api-gateway (port 8080)
- Eureka does not register with itself (`register-with-eureka=false`, `fetch-registry=false`)

7.2 API Gateway

The API Gateway (Spring Cloud Gateway, port 8080) is the single entry point for all client traffic. It handles:

- Route resolution — maps URL path patterns to upstream services via Eureka load-balanced URIs.
- Rate limiting — the `/api/auth/**` routes use Redis-backed `RequestRateLimiter` (10 requests/second, burst of 20) keyed by client IP to prevent brute-force attacks on login/register.
- CORS — a global CORS filter allows the React frontend (`http://localhost:5173`) with credentials and all methods. A `DedupeResponseHeader` filter removes duplicate CORS headers that some services might also set.
- JWT forwarding — the Gateway validates the JWT and injects `X-User-Id` and `X-Roles` headers before forwarding requests to downstream services. Services trust these internal headers instead of re-validating the token.
- BFF Dashboard — the Gateway locally hosts `DashboardController` which aggregates data from IAM and Academic Core using reactive `WebClient` calls in parallel (`Mono.zip`), returning a single merged response to the frontend.
- WebSocket passthrough — `/ws/**` routes are forwarded to the Communication Service for STOMP WebSocket connections.

7.3 Gateway Route Configuration

The table below shows all configured routes from `application.properties`:

Route ID	Rule
iam-service-auth (Rate Limited)	Path= <code>/api/auth/**</code> → <code>lb://IAM-SERVICE</code> . <code>RequestRateLimiter</code> : 10 req/s, burst 20. Key by IP address.

Route ID	Rule
iam-service-auth (Rate Limited)	Path=/api/auth/** → lb://IAM-SERVICE. RequestRateLimiter: 10 req/s, burst 20. Key by IP address.
iam-service-other	Path=/api/users/**, /api/students/**, /api/teachers/** → lb://IAM-SERVICE
academic-core	Path=/api/courses/**, /api/departments/**, /api/enrollments/**, /api/enrolled-courses/**, /api/announcements/**, /api/feedbacks/**, /api/audit-logs/** → lb://ACADEMIC-CORE
communication-service	Path=/api/notifications/**, /api/messages/** → lb://COMMUNICATION-SERVICE
communication-service-ws	Path=/ws/** → lb://COMMUNICATION-SERVICE (WebSocket upgrade passthrough)
BFF dashboard (local)	GET /api/gateway/dashboard/** handled locally by DashboardController — no routing needed.

8. Aspect-Oriented Programming (AOP)

AOP is used across three services to handle cross-cutting concerns — logging, security enforcement, and audit recording — without polluting business logic classes. All aspects use Spring AOP with AspectJ annotations.

8.1 Aspects Summary

Service	Aspect Class	Advice Type	Responsibility
Academic Core	AuditLogAspect	@AfterReturning + @annotation(auditLog)	Intercepts methods annotated with @AuditLog. Reads X-User-Id, X-Username and X-Roles from the request headers forwarded by the Gateway, then calls AuditLogRecordingService to persist a structured audit entry.
Academic Core	TeachersOnlyAspect	@Before + @annotation(teachersOnly)	Runs first (Order 1). Reads the X-Roles header; throws 403 if the caller is not TEACHER or ADMIN. Guards all write operations on courses and announcements.
Academic Core	CourseTeacherOnlyAspect	@Before + @annotation(courseTeacherOnly)	Runs second (Order 2). Resolves the courseId from @PathVariable or request body, loads the course, then verifies the caller's X-User-Id matches the course's teacherId. Admins bypass this check.
Communication	LoggingAspect	@Before, @AfterReturning, @AfterThrowing, @Around	Covers service, controller, repository and Kafka layers. Logs method entry/exit, return type, exceptions, and measures execution time. Warns if any service method takes over 1000 ms.
IAM	LoggingAspect	@AfterReturning,	Applies to every public

Service	Aspect Class	Advice Type	Responsibility
		@AfterThrowing, @Around	method in com.uni.iam.service.**. Logs success return types, exception details, and execution duration without touching any business logic class.

9. Design Patterns

The following design patterns are used across the system. Each entry identifies the pattern, exactly where in the codebase it appears, and how it is applied.

Service	Pattern	Location in Code	How It Is Applied
Academic Core	Hexagonal Architecture (Ports & Adapters)	Infrastructure layer	The domain is completely isolated from frameworks. Inbound adapters (HTTP controllers) call domain use case ports (interfaces). Outbound adapters implement repository/event ports: CoursePersistenceAdapter, EnrollmentPersistenceAdapter, FeedbackPersistenceAdapter, AnnouncementPersistenceAdapter (persistence) and KafkaEventPublisher (messaging). The domain never imports Spring.
Academic Core	Repository / Adapter (Outbound)	infrastructure/adapters/out/persistence/	Each persistence adapter implements a domain port interface (e.g. CourseRepositoryPort). It maps between domain models and JPA entities using a dedicated PersistenceMapper, keeping JPA annotations out of the domain model.
Academic Core	Cache-Aside (Spring @Cacheable)	CoursePersistenceAdapter	Read methods (findAll, findById, findPopular, findByTeacherId, findByDepartmentName) are annotated with @Cacheable. Write/delete methods use @CacheEvict across all 7 course cache regions to maintain consistency.

Service	Pattern	Location in Code	How It Is Applied
Academic Core	Feign Client + Circuit Breaker (Fallback)	IamClient / IamClientFallbackFactory	The Academic Core calls IAM (for teacher/student names) via a declarative Feign client. IamClientFallbackFactory provides graceful degradation when IAM is unavailable.
IAM	Facade	StudentServiceImpl.getStudentDetails()	A single method orchestrates calls to StudentRepository, TeacherRepository, and AcademicCoreClient (for enrollments and announcements), then assembles the complete StudentProfileResponse. The caller sees one simple call; the complexity of coordinating three data sources is hidden behind the service facade.
IAM	Strategy — Academic Standing	StudentServiceImpl.determineAcademicStanding()	A private method encapsulates interchangeable GPA-based classification logic: GPA >= 3.5 → Excellent, GPA >= 2.5 → Good Standing, else → Probation. The strategy is applied uniformly at profile-build time, making it easy to swap or extend classification rules without modifying the calling code.
IAM	Template Method (Inheritance)	User / Student / Teacher entities	User defines the common fields (id, username, email, role, createdAt) using @MappedSuperclass + @SuperBuilder. Student and Teacher extend it adding academic-specific fields (gpa, totalCredits, enrollmentDate / officeLocation, department). JPA uses JOINED table strategy.
API Gateway	Backend-for-Frontend (BFF)	DashboardAggregationService	The Gateway hosts a BFF layer that makes parallel reactive calls (Mono.zip) to IAM and Academic Core, merges the results, and returns a single optimised DTO per role (StudentDashboardResponseDto,

Service	Pattern	Location in Code	How It Is Applied
All services	Dependency Injection / IoC (Spring)	Every @Service, @Component, @Repository	All dependencies are injected via constructor injection (@RequiredArgsConstructor). Interfaces are used everywhere so implementations can be swapped without changing callers (SOLID DIP).

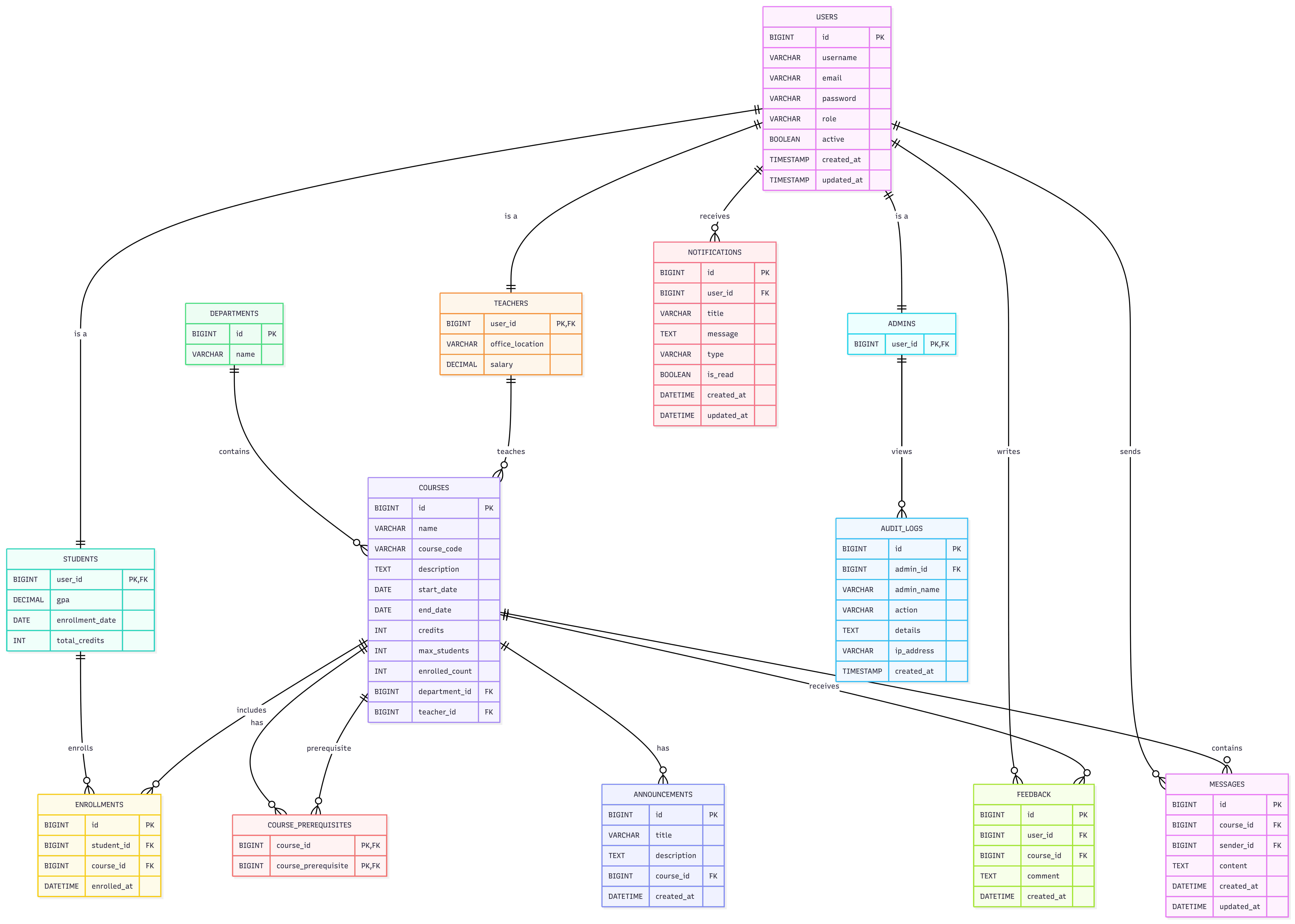
10. Most Important API Endpoints

The table below lists the 21 most critical endpoints — the ones the frontend depends on most heavily, or that encode the most important business rules.

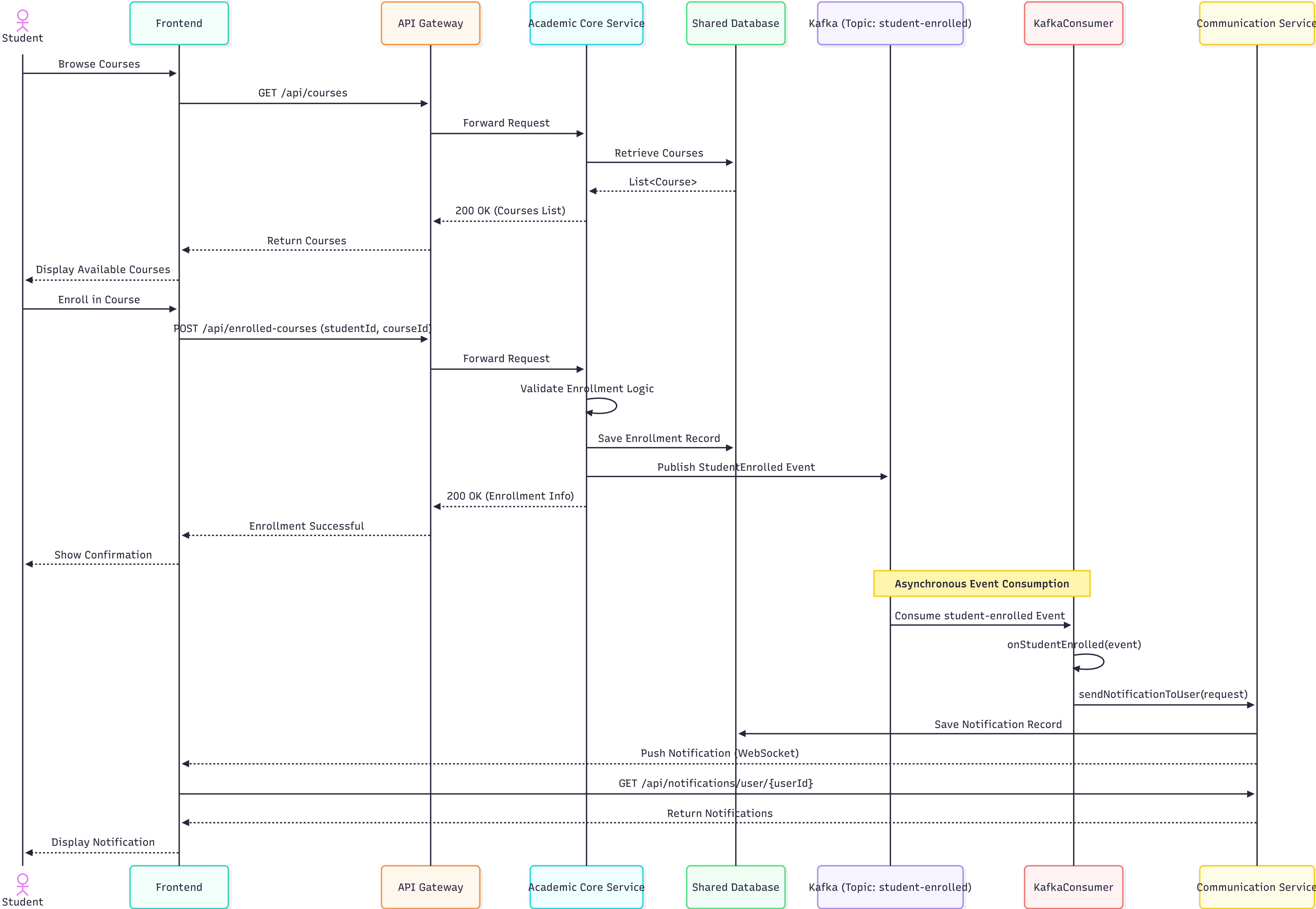
Service	Method	Endpoint	Description
IAM	POST	/api/auth/register	Register a new user. Body includes username, email, password, and role (STUDENT or TEACHER). Returns a JWT token on success. Rate-limited at the Gateway (10 req/s, burst 20).
IAM	POST	/api/auth/login	Authenticate an existing user. Returns a signed JWT that must be sent as Bearer in the Authorization header for all subsequent requests.
IAM	GET	/api/users/me	Returns the profile of the currently authenticated user by extracting the userId from the JWT. Used by the frontend on startup.
IAM	GET	/api/students/details/{id}	Returns the full student profile: GPA, total credits, academic standing, list of enrolled courses (fetched from Academic Core via Feign), and upcoming events. Powers the student dashboard.
IAM	GET	/api/teachers/details/{id}	Returns the full teacher profile including courses taught (fetched from Academic Core). Powers the teacher dashboard.
Academic Core	POST	/api/courses	Create a new course. Requires Teacher role (enforced by @TeachersOnly AOP). Body includes name, courseCode, description, maxStudents, credits, departmentId, teacherId, startDate, endDate. Triggers a

Service	Method	Endpoint	Description
			Kafka event.
Academic Core	GET	/api/courses/all	Returns all courses. Response is cached (COURSES_ALL_CACHE). Used to populate the course catalogue on the frontend.
Academic Core	GET	/api/courses/popular	Returns the top-N most-enrolled courses ordered by enrolledCount descending. Used for the homepage discovery section.
Academic Core	GET	/api/courses/Department/{name}	Filters courses by department name (case-insensitive). Resolves the department entity first, then queries courses by departmentId.
Academic Core	POST	/api/enrolled-courses	Enroll the authenticated student in a course. Returns 201 on success. Triggers a Kafka notification event to the Communication service.
Academic Core	DELETE	/api/enrolled-courses/drop	Drop a course for a student. Accepts studentId and courseId as query params. Triggers a Kafka event.
Academic Core	GET	/api/enrolled-courses/student/{id}	Returns all enrollments for a given student. Used by the BFF to build the student dashboard.
Academic Core	POST	/api/announcements/create	Teacher creates an announcement for a course. Requires Teacher role (AOP-enforced). Triggers a Kafka notification to all enrolled students.
Academic Core	GET	/api/announcements/student/{id}	Returns all announcements across all courses the student is enrolled in. Fetches enrollments first, then aggregates announcements per course.
Communication	GET	/api/notifications/user/{id}/unread	Returns all unread notifications for a user. Used to populate the notification bell on the frontend.
Communication	PATCH	/api/notifications/user/{id}/read-all	Marks all notifications for a user as read. Returns the count of updated records.
Communication	POST	/api/notifications/course	Sends a notification to every student enrolled in a course.

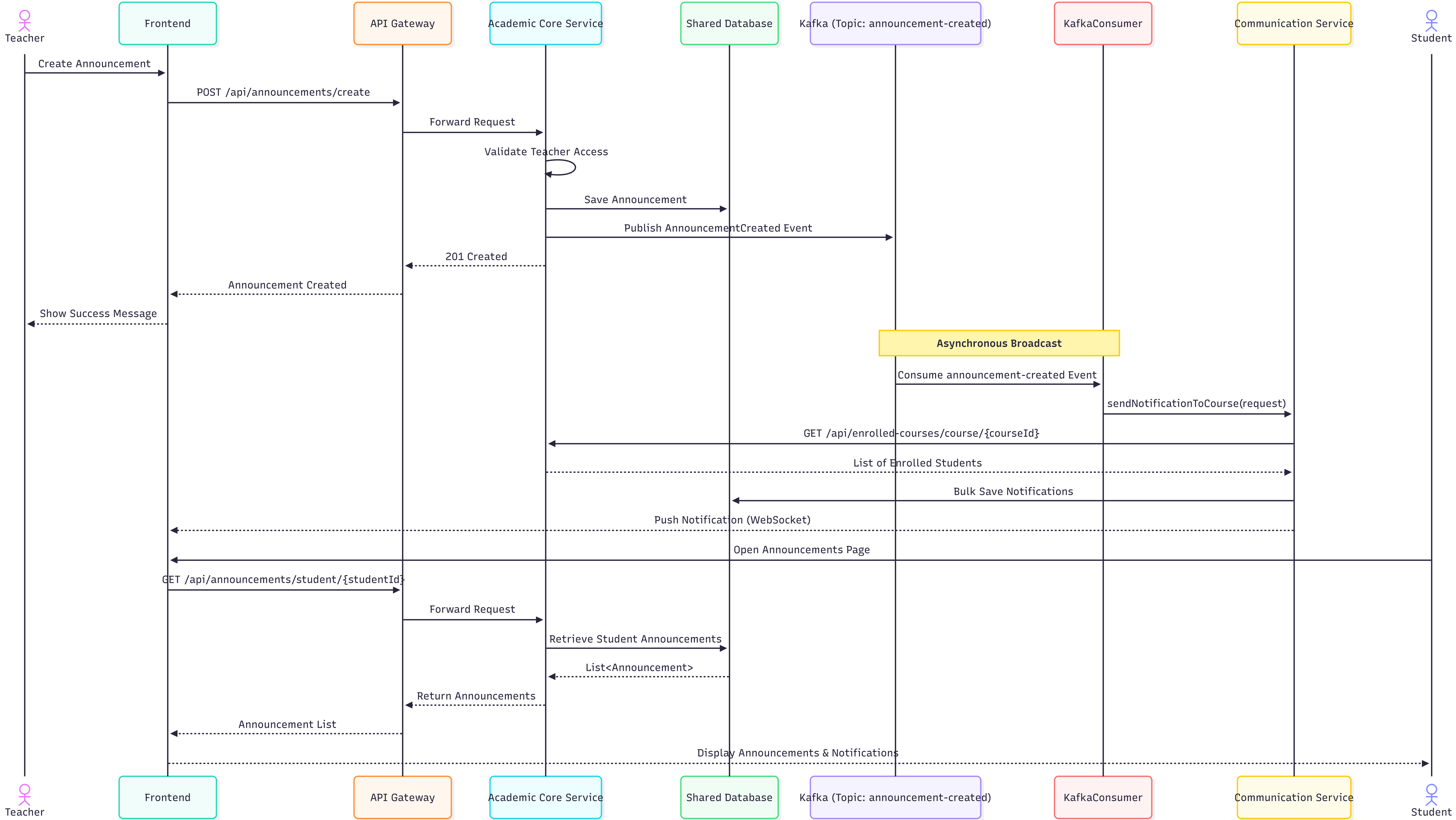
Service	Method	Endpoint	Description
			Used internally by Kafka consumers after enrollment/announcement events.
Communication	GET	/api/messages/course/{courseId}	Returns all chat messages for a course ordered chronologically. Called on page load to hydrate the chat history.
Communication	WS	/app/course/{courseId}	STOMP WebSocket endpoint. Clients send a MessageRequest; the server persists it and broadcasts it to /topic/course/{courseId} so all subscribers receive it in real time.
API Gateway	GET	/api/gateway/dashboard/user	Reads the JWT, identifies the user's role, then calls either the student or teacher dashboard aggregation path. Returns a single merged DTO for the frontend home page.
API Gateway	GET	/api/gateway/dashboard/student/{id}	BFF: calls IAM for the student profile and Academic Core for enrollments in parallel (Mono.zip), merges results into StudentDashboardResponseDto.



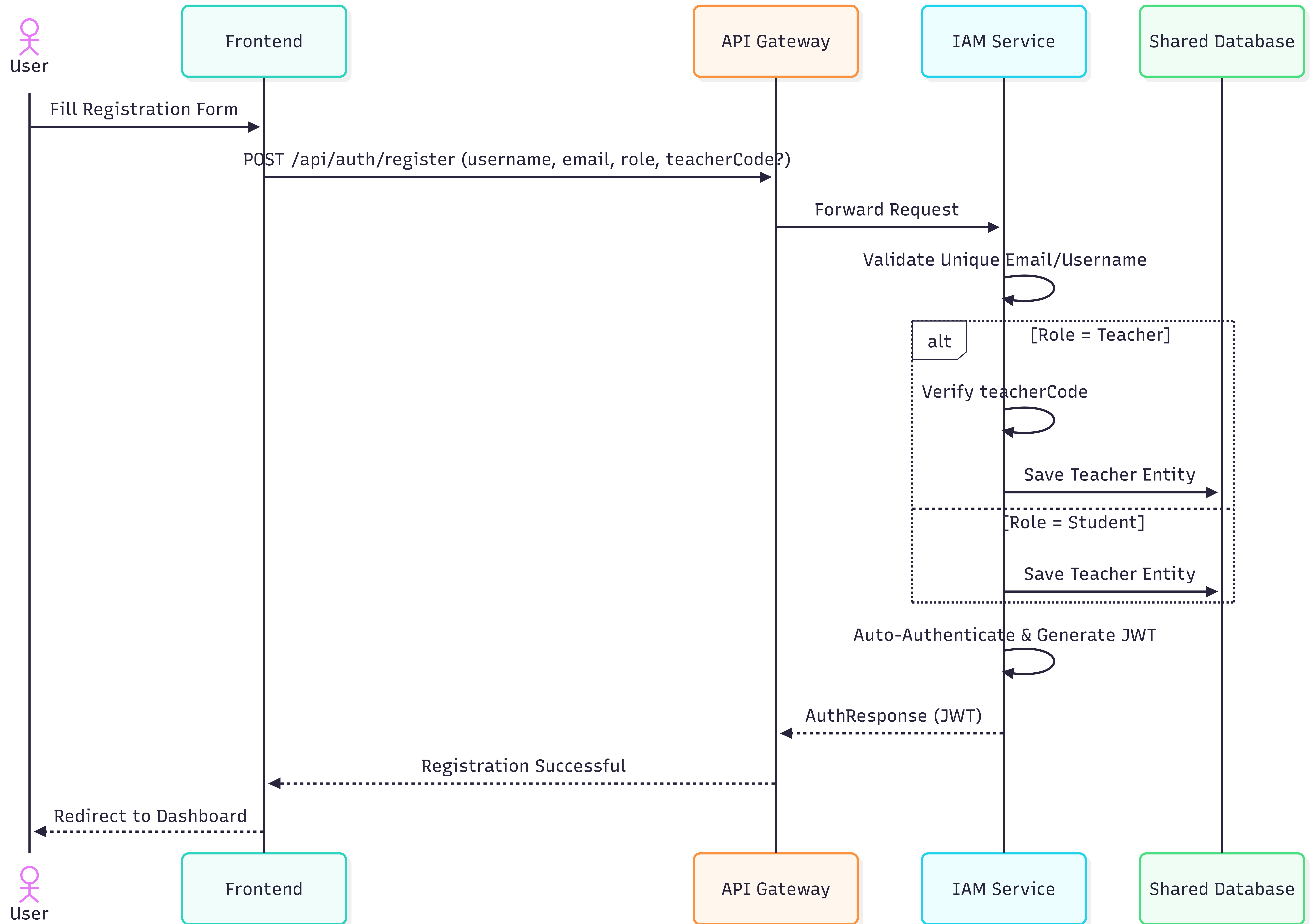
2. Student Course Enrollment

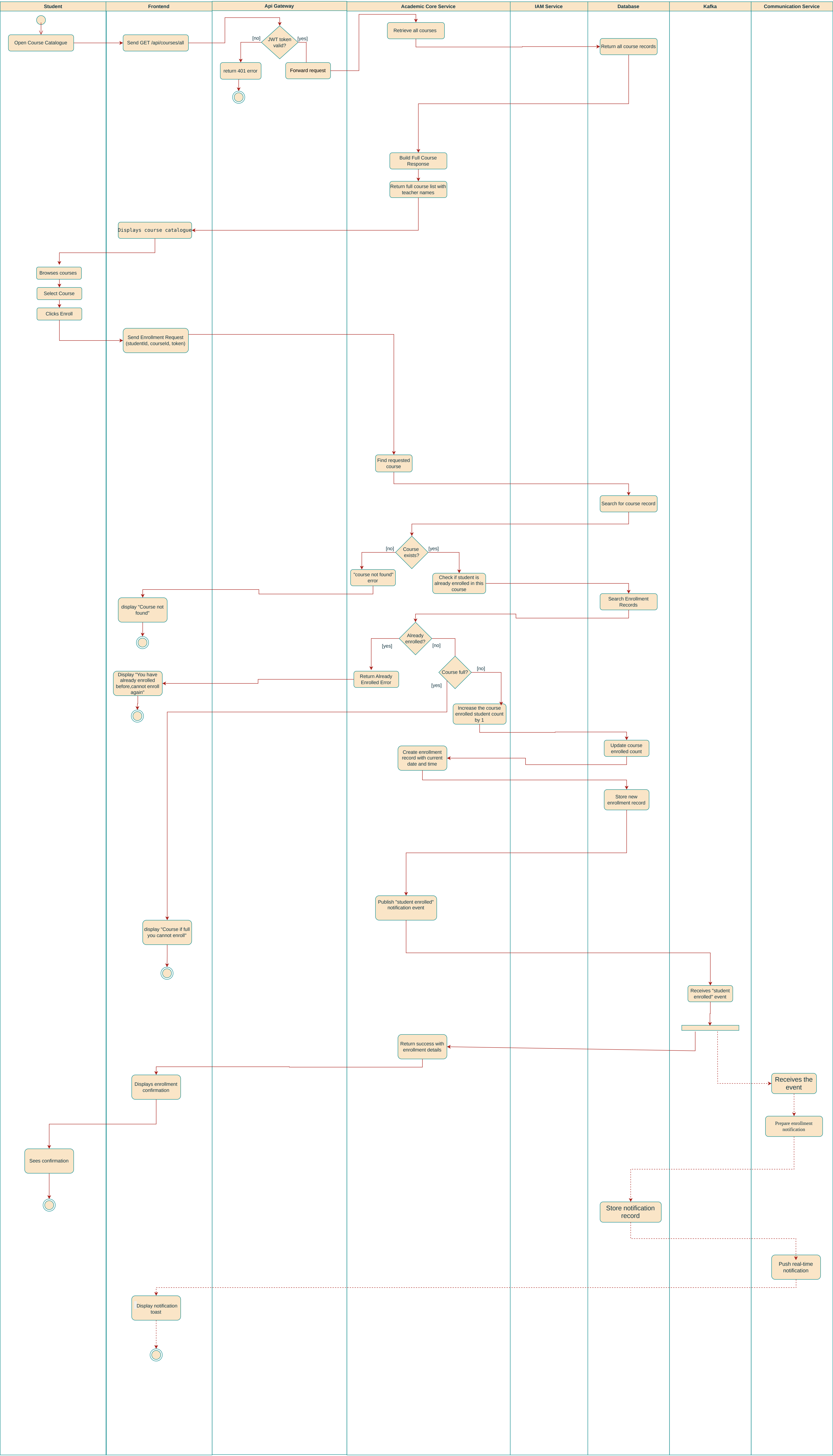


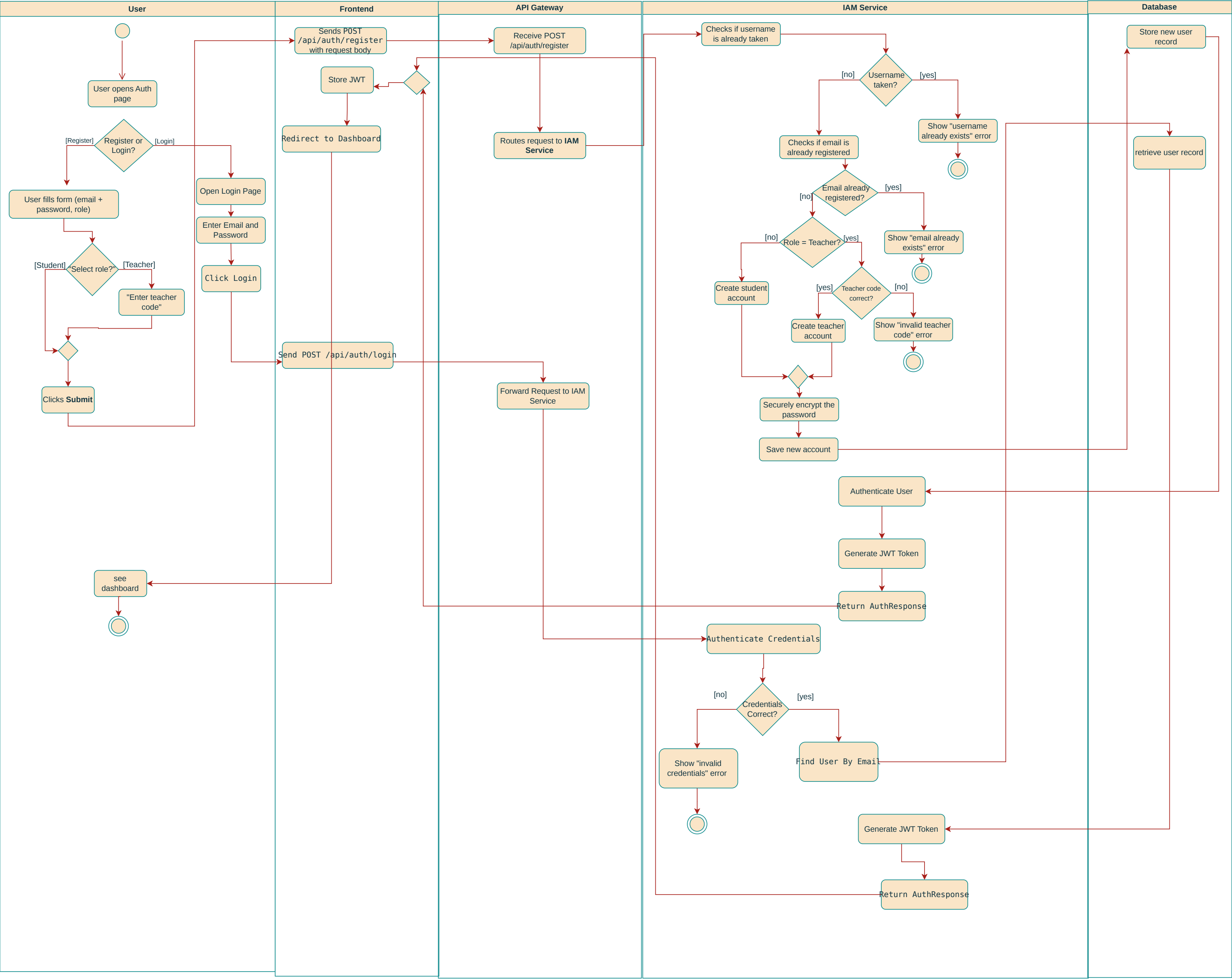
4. Teacher Posting Announcement (Broadcast Flow)

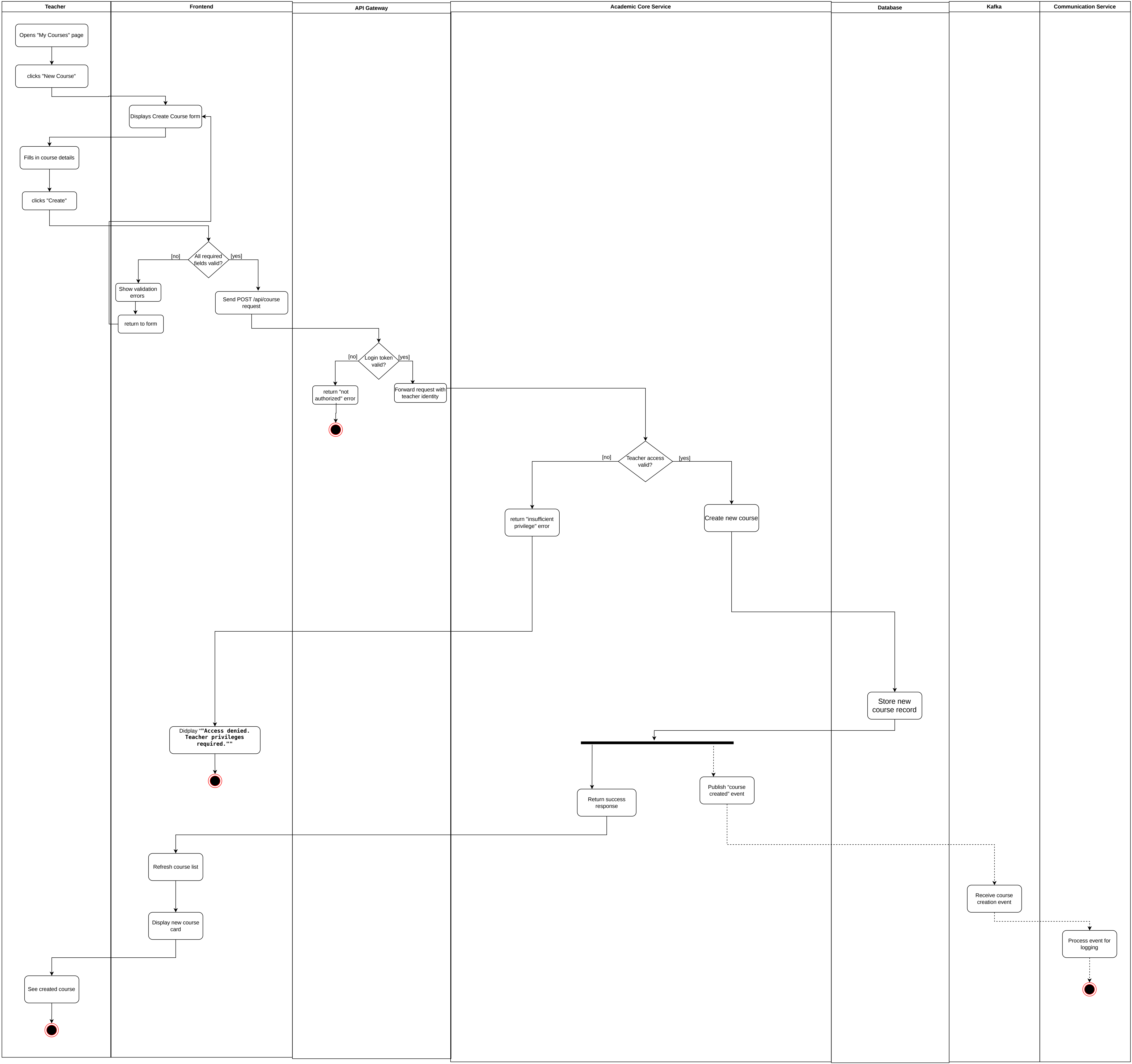


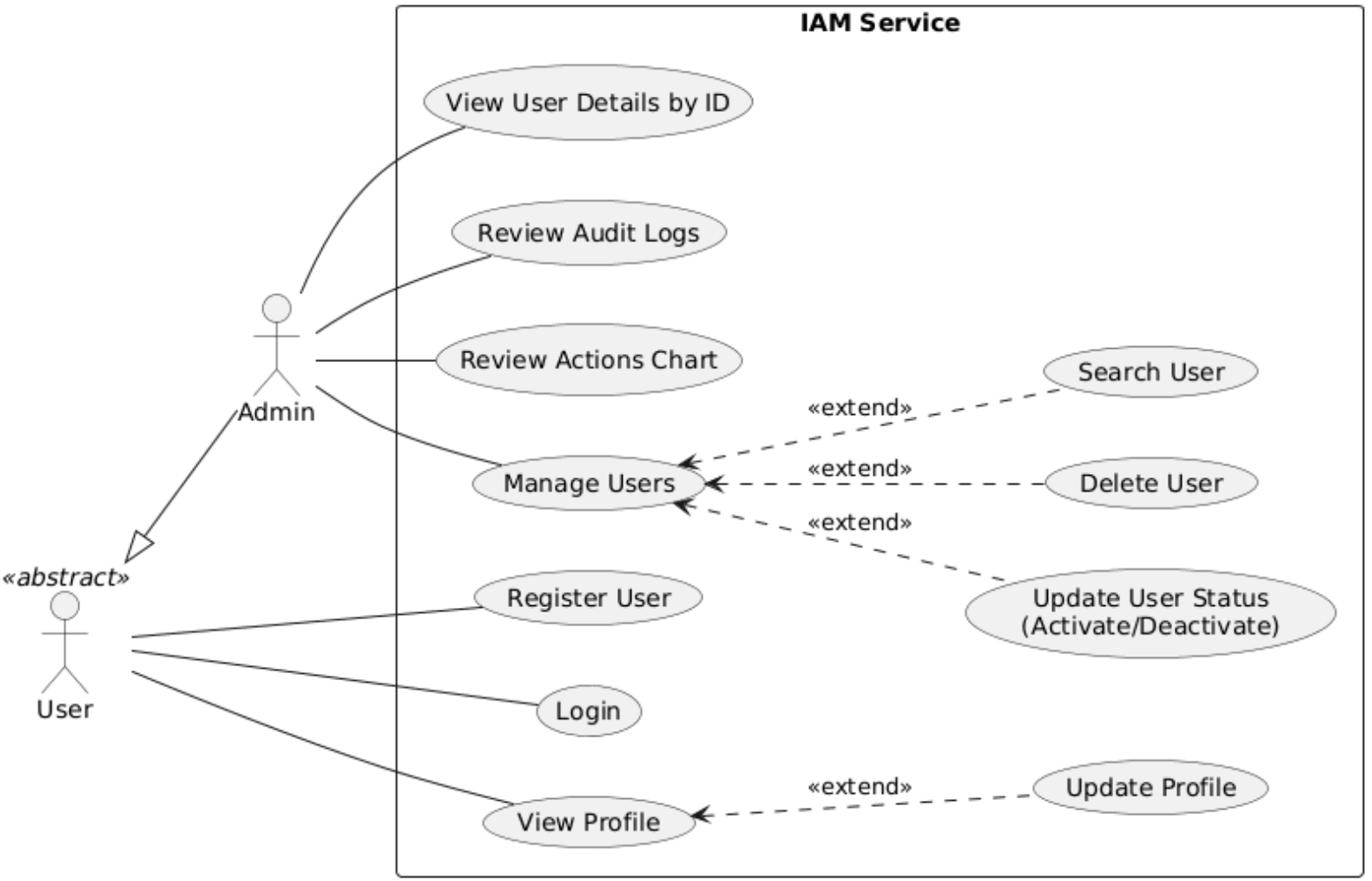
1. User Registration & Profile Logic

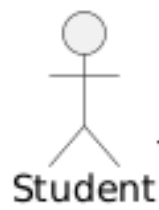












Communication Service

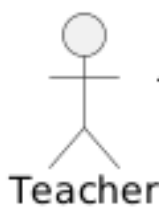
Mark All Notifications as Read

«extend»

Mark Notification as Read

«extend»

View Notification Details



Academic Core Service

Student Use Cases

Drop Course

View Own Enrollments

Submit Course Feedback

View All Courses

Enroll in Course

Extended Use Cases

View Course Details

View Enrolled Students

Send Message

Teacher Use Cases

Unenroll Students from Own Courses

Create New Course

View Own Courses

Create Course Announcement

View Feedback

Admin Use Cases

Departments Management

Courses Management

Student

Teacher

Admin

«extend»

«extend»

«extend»

«extend»

«extend»

«extend»

«extend»

